



System Architecture for Governed MDA API Interoperability

A visual architecture submission showing platform components, data flow, security boundaries, user workflows, API governance, sandbox execution, and production-readiness path for Uganda GovHub API.

THEMATIC AREA

Area 10: Interoperability and Data Exchange

SUBMISSION SCOPE

Working prototype using deterministic sandbox records

PREPARED FOR

MoICT&NG Prototype Showcase

What the system is designed to do

Uganda GovHub API is a developer portal, sandbox, and governance console for Ministries, Departments, and Agencies. It demonstrates how government APIs can be discovered, documented, requested, approved, tested, and audited without exposing live citizen records.

5

SEEDED SHOWCASE APIS

4OPERATIONAL DEMO
ROLES**10**PLATFORM
SCREENSHOTS**0**

LIVE PRODUCTION CALLS

Current implementation

The prototype is a local full-stack web application. The frontend is React, TypeScript, Vite, React Router, and GovHub UI components. The backend is Node.js, Express, TypeScript, OpenAPI YAML files, and PostgreSQL through the `pg` driver.

- Routes include catalog, docs, API detail, dashboard, settings, and API registration.
- Seeded accounts represent developer, API owner, compliance reviewer, and platform administrator roles.
- PostgreSQL stores users, sessions, MDAs, API records, access requests, audit logs, API keys, rate limits, and verification metadata.

Government deployment intent

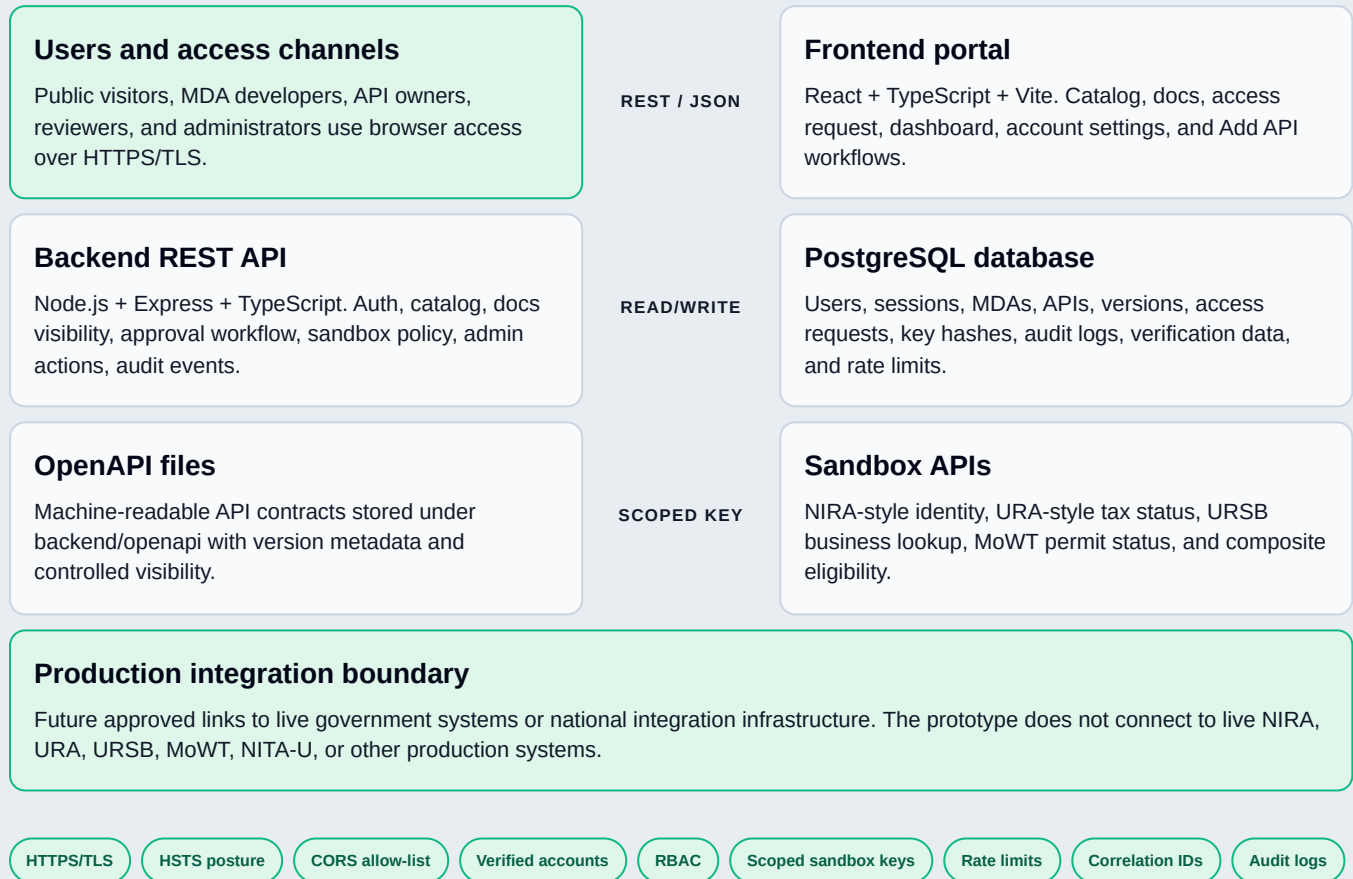
The architecture separates public discovery from operational access. It places policy metadata next to API contracts, requires account and API-level approval, scopes sandbox credentials, and records audit events with correlation IDs.

- Mock records keep the showcase repeatable and safe.
- Live NIRA, URA, URSB, MoWT, and NITA-U systems are not called in the prototype.
- The design can be hardened for production without replacing the core governance workflow.

This submission follows the Ministry guidance by showing the main components, communication paths, storage and security controls, user access routes, APIs and integrations, hosting posture, screenshots of key workflows, and practical readiness considerations.

System components and communication paths

Users reach the React portal through a browser. The portal calls the Express backend over REST/JSON. PostgreSQL holds application state, OpenAPI files hold machine-readable contracts, and sandbox handlers simulate approved government API calls.



Architecture components requested by the guidance

The system presents the main parts in a way a technical evaluation panel can review for fitness, interoperability, security, maintainability, and transferability.

COMPONENT	IMPLEMENTED DESIGN	WHAT IT SHOWS FOR EVALUATION
Frontend / user interface	React, TypeScript, Vite, dashboard UI components, API docs pages, account pages, and catalog workflows.	Browser access for public discovery, verified account operations, requests, approvals, audit review, and API onboarding.
Backend / application layer	Node.js, Express, TypeScript routes, middleware, policy helpers, OpenAPI validation, docs visibility, access control, audit service, and sandbox handlers.	Business logic is centralized behind REST endpoints with auth, role, visibility, sandbox scope, rate-limit, and audit controls.
Database layer	PostgreSQL through `pg`; local demo can run in a postgres:16-alpine container and deployment can use hosted Postgres.	Data categories and relationships are explicit, transferable, and suitable for a production relational database path.
Authentication and access	Signup/login, account review states, seeded demo accounts, role-based access, MFA support, bearer sessions, scoped API keys, expiry, revocation, and docs visibility rules.	Public transparency is separated from operational trust; developers, owners, reviewers, and admins have distinct permissions.
APIs and integrations	Catalog and docs APIs plus sandbox endpoints for identity, tax, business registration, driving permit, and composite eligibility scenarios.	Documented APIs are exposed; prototype integrations use internal mock handlers only.
Security controls	TLS/HSTS hooks, CORS allow-listing, body limits, password hashing, encrypted sensitive fields, key hashing, rate limits, correlation IDs, redacted logs, and structured denial responses.	Security is applied across access, storage, transport, API execution, and audit review.

Interoperability catalog

The catalog gives MDAs one place to discover API products across agencies, filter by status and compliance, and begin a governed access workflow.

Uganda GovHub API
Developer Portal

Dashboard

API Catalog

API Docs

ALL SERVICES ONLINE

Settings

Access Groups

Get Help

Platform Admin
system.admin@govh...

Uganda GovHub API / API Catalog

Admin

MoICT

Search catalog...

Interoperability Catalog

Discover, test, and request lawful access to secure data sharing APIs owned by Ugandan Ministries, Departments, and Agencies (MDAs).

Search by name, sector, agency...

All Lifecycles

All Compliance

+ Add API

Transport

BETA

MEDIUM

Driving Permit Verification API

Verify driving permit status and validity class.

OwnerMoWT

Identity

PRODUCTION

HIGH

NIRA Identity Verification API

Verify citizen identity using NIN. Returns match confidence and basic verification status.

OwnerNIRA

Integration

DRAFT

HIGH

Service Uganda Composite Eligibility

Composite workflow checking citizen identity, tax status, and driving permit in a single call.

OwnerMoICT

Finance

PRODUCTION

MEDIUM

URA Tax Compliance Status API

Check tax clearance status for businesses and individuals.

OwnerURA

Commerce

BETA

LOW

URSB Business Registration Lookup

Look up business registration details and verify company status.

OwnerURSB

CATALOG

Seeded APIs from NIRA, URA, URSB, MoWT, and MoICT are discoverable by sector, lifecycle, compliance state, and sensitivity.

User access

Browser-based portal with role-aware shell and platform identity state.

Discovery

Catalog cards expose owner, sector, lifecycle, and sensitivity signals.

Operations

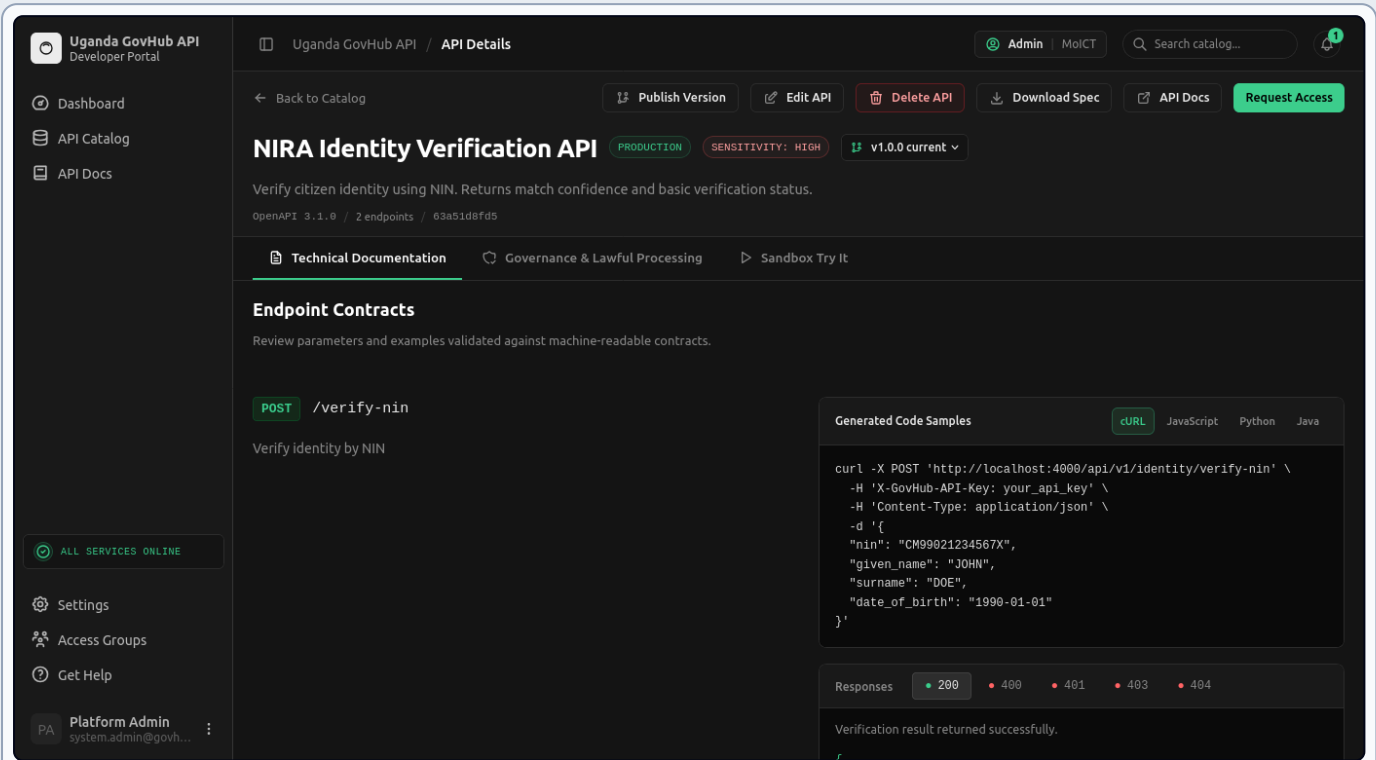
Search, filters, Add API action, and service-health state are visible.

Screenshot evidence: API catalog

Page 5 of 18

API detail and technical contract

API detail pages place documentation, governance metadata, sandbox access, lifecycle controls, version publishing, and downloadable specifications in one workflow.



API DETAIL

The NIRA Identity Verification API shows lifecycle status, sensitivity, OpenAPI versioning, code samples, and access-request entry points.

Contracts

Endpoint methods and sample requests are generated from the API definition.

Governance

Sensitivity and lifecycle labels make policy state visible beside the technical contract.

Control

Publish, edit, delete, download, docs, and request-access actions are role-aware.

Restricted API documentation

Documentation pages explain endpoint purpose, security requirements, server URLs, access requirements, and integration flow while preserving visibility restrictions for sensitive APIs.

Uganda GovHub API / API Docs

AdminMolCT

Search catalog...

Back to API Docs

Edit docsDownload OpenAPI

NIRA Identity Verification API

Restricted docs

OPENAPI 3.1.0

V1.0.0

Verify citizen identity using the National Identification Number (NIN). This API returns a match confidence score and basic verification status to comply with the Data Protection and Privacy Act, 2019. It does not return the full citizen profile.

Restricted docs. This documentation is restricted to administrators, compliance reviewers, owning MDA API owners, and consumers with approved access. Sensitivity is High and classification is Restricted.

ENDPOINTS

Verification

POST /verify-nin

Status

GET /status/{nin}

API Guide

Start here to understand purpose, access, servers, endpoints, inputs, and responses. The OpenAPI contract is still the source of truth; this guide explains how to use it.

Who This Is For

For approved access groups working with sensitive MDA data.

Access Requirements

Runtime use needs an approved request and active API key. API owners and reviewers can inspect the contract.

Integration Flow

Review purpose, confirm fields, test sandbox if available, request access, configure the key, and log correlation IDs.

Servers

Use these base URLs for requests. Prefer sandbox while building and production only after approval.

https://sandbox.govhub.go.ug/api/v1/identity

Sandbox Environment

Security

Docs visibility controls who can read this page. API keys control who can call protected endpoints.

ApiKeyAuth

apiKey - header - X-GovHub-API-Key

API DOCS

Restricted documentation is visible to authorized roles and approved consumers, with access requirements and integration flow presented beside the API contract.

Visibility

Restricted documentation is not treated as a public page for all users.

Integration

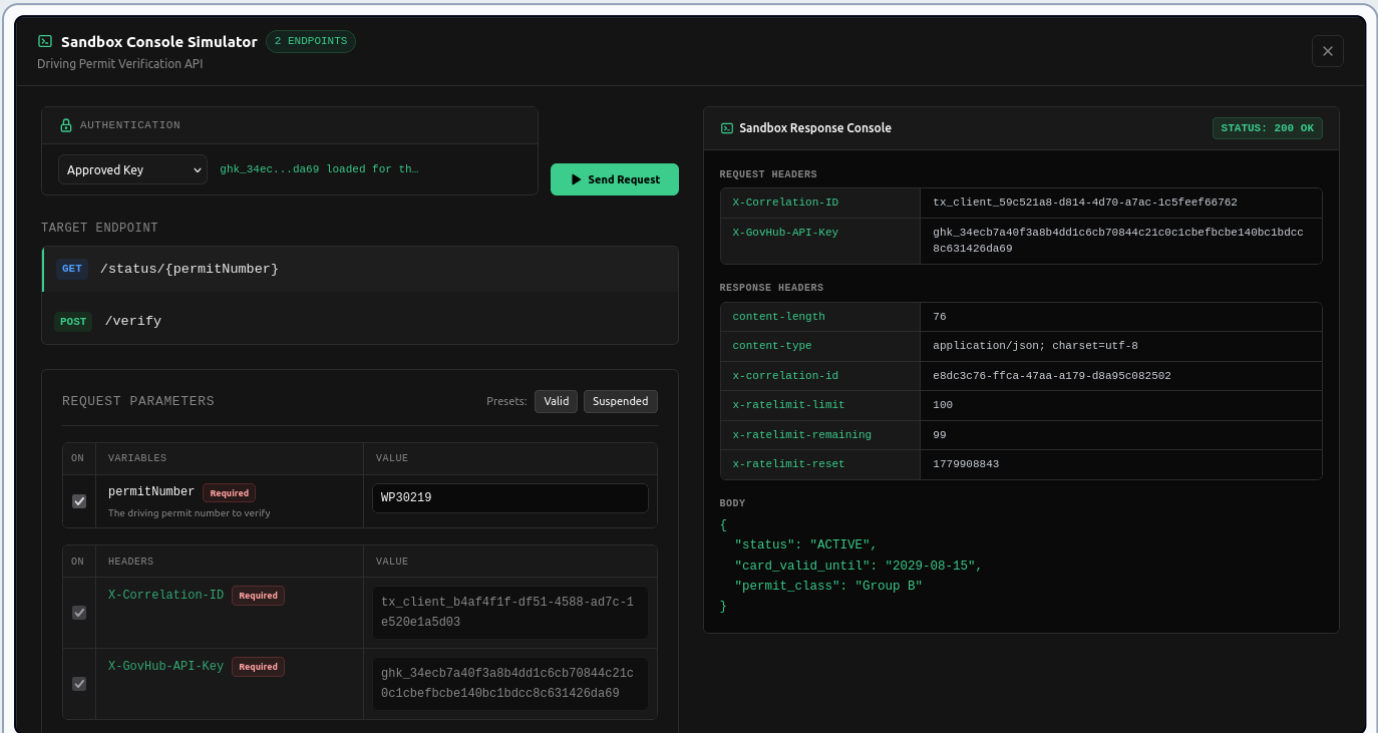
Server, security scheme, endpoint, and guide sections are visible to implementers.

Traceability

Docs explain that protected calls require approved access and API keys.

Sandbox console simulator

The sandbox console demonstrates scoped API key enforcement, approved-key selection, request parameters, response headers, rate-limit metadata, and correlation IDs.



SANDBOX

The simulator sends requests using an approved key and returns mock responses with request headers, response headers, rate-limit values, and `x-correlation-id` traceability.

Scope check

Sandbox requests require a loaded approved key for the target API.

Safe data

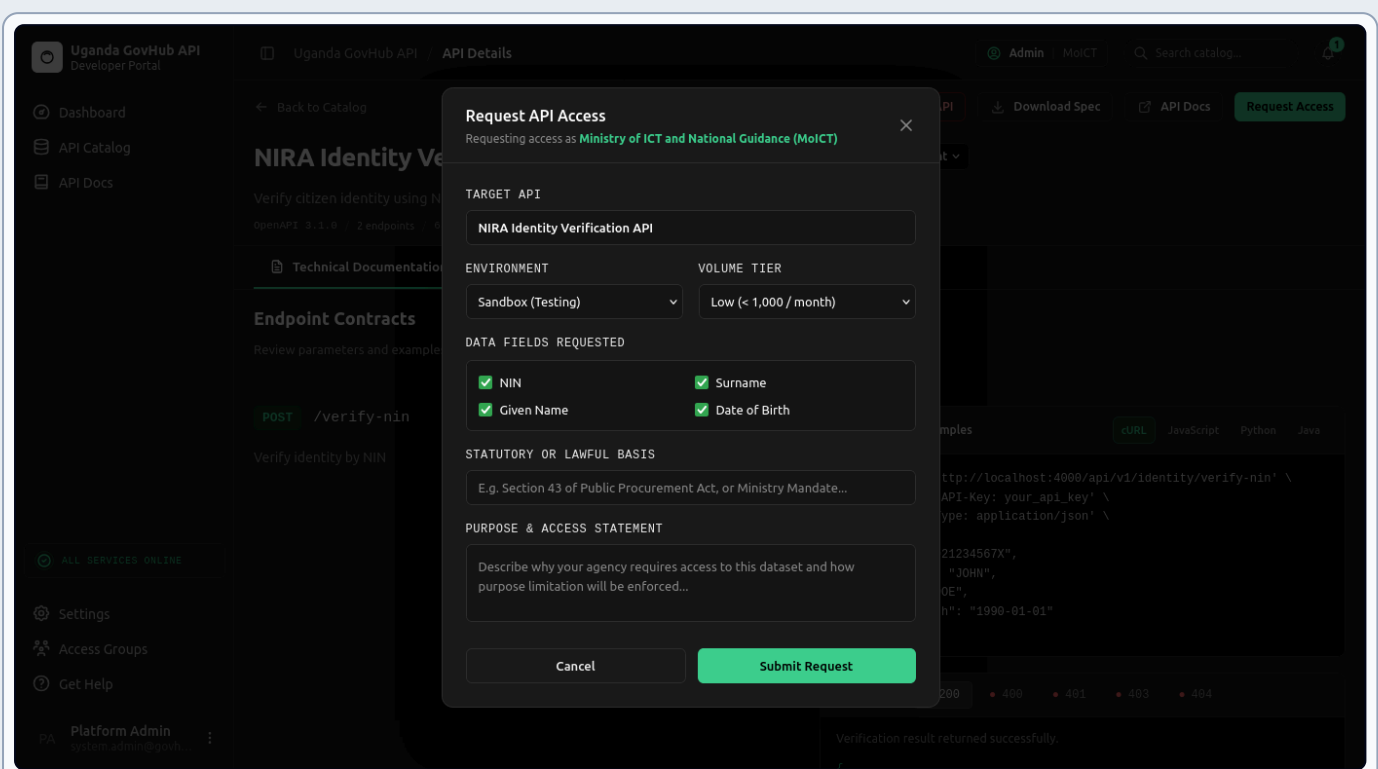
Responses are deterministic mock outputs, not live government data.

Auditability

Correlation IDs and rate-limit headers support operational traceability.

Access request workflow

Consumers request API access with explicit purpose, lawful basis, environment, volume tier, and requested data fields so data sharing is reviewable before credentials are issued.



REQUEST

The request form captures target API, environment, volume tier, requested data fields, statutory basis, and purpose statement.

Purpose limitation
Business purpose and statutory basis are mandatory review inputs.

Data minimization
The requester selects fields rather than receiving all available data.

Environment control
Sandbox access is separated from any future production access path.

Approval and key lifecycle

Owners or administrators can review requests, approve scoped keys, set expiry, and manage pending or active access from the operational dashboard.

Uganda GovHub API
Developer Portal

Dashboard

API Catalog

API Docs

ALL SERVICES ONLINE

Settings

Access Groups

Get Help

Platform Admin
system.admin@govh...

Uganda GovHub API / Dashboard

Admin

MolCT

Search dashboard...

APPROVED CHANNELS
1

PENDING APPROVALS
1

TOTAL AUDITED HITS
3

COMPLIANCE RATE
67%

Access Approvals 1

Accounts 4

My Agency Credentials

Audit Trails

Interoperability Matrix

Analytics

Active Access Requests

Evaluate legal mandate alignment and manage cryptographically bound sandbox API keys.

CONSUMER MDA	API REQUESTED	LAWFUL BASIS	PURPOSE	FIELDS & TIER	STATUS	ACTION
Ministry of Health	URSB Business Registration Lookup	"test"	test	Low (< 1,000 / month) BRN, Company Name, Dir...	PENDING	Jun 26, 2026, 10:01 PM Approve key
Ministry of Health	Driving Permit Verification API	"test"	test	Low (< 1,000 / month) Permit Number, Class, Ex...	ACTIVE	ghk_34ec...da69 Jun 26, 2026, 7:50 PM

APPROVE

The dashboard exposes pending and active requests with consumer MDA, API, basis, purpose, field tier, status, expiry, and action controls.

Governance

Reviewers see legal basis, purpose, field selection, and status before approval.

Credential control

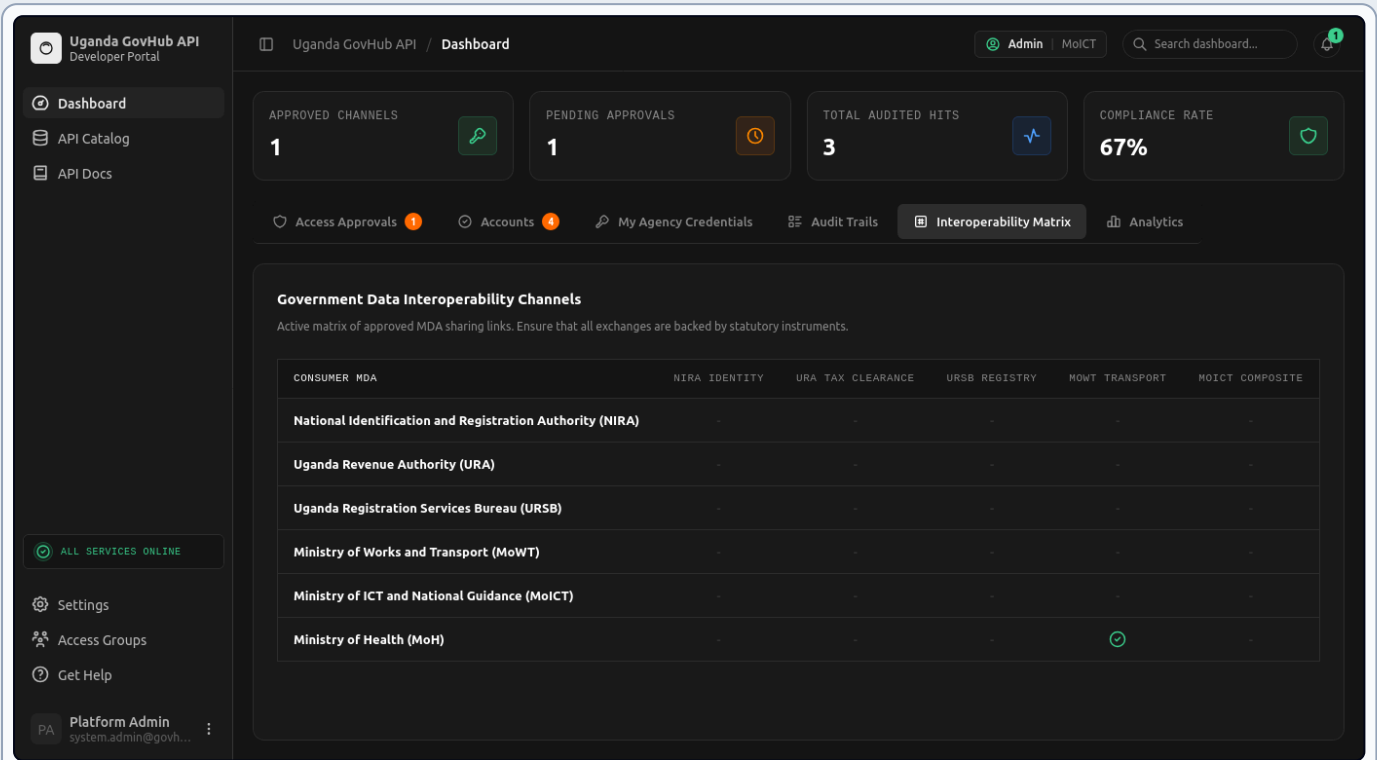
Issued keys are scoped, expiring, and manageable from the dashboard.

Oversight

Dashboard counters expose approved channels, pending approvals, audited hits, and compliance rate.

Government interoperability matrix

The matrix gives MoICT and reviewers a ministry-level map of approved data-sharing channels between consuming MDAs and source APIs.



MATRIX The interoperability matrix gives a ministry-level view of approved MDA data-sharing channels across source API products.

Interoperability map

Shows who is approved to consume which government API.

Audit support

Approved channels can be compared against audit-log activity.

Risk visibility

Inactive or missing channels are visually distinct from approved relationships.

OpenAPI import and validation

Authorized users can register new APIs by importing OpenAPI specifications from a URL, uploaded file, or raw code before completing governance metadata.

Uganda GovHub API
Developer Portal

Dashboard

API Catalog

API Docs

ALL SERVICES ONLINE

Settings

Access Groups

Get Help

Platform Admin
system.admin@govh...

Uganda GovHub API / Add API

Admin MoICT Search catalog...

Cancel

Back to API Catalog

Register New API

Import an OpenAPI document, validate it, then complete the governance metadata for registry activation.

Import the API OpenAPI Specification file. The GovHub validation engine will extract structure and endpoints to prepare compliance sheets.

Spec URL

Upload Spec

Raw Code

OPENAPI URL

https://raw.githubusercontent.com/OAS/main/spec.yaml

Validate OpenAPI Specification

IMPORT

Spec URL, upload, and raw-code modes support OpenAPI onboarding without manually rebuilding endpoint documentation.

Onboarding

New MDA APIs can enter the registry through a standardized workflow.

Contract-first

OpenAPI validation happens before the catalog record is created.

Maintainability

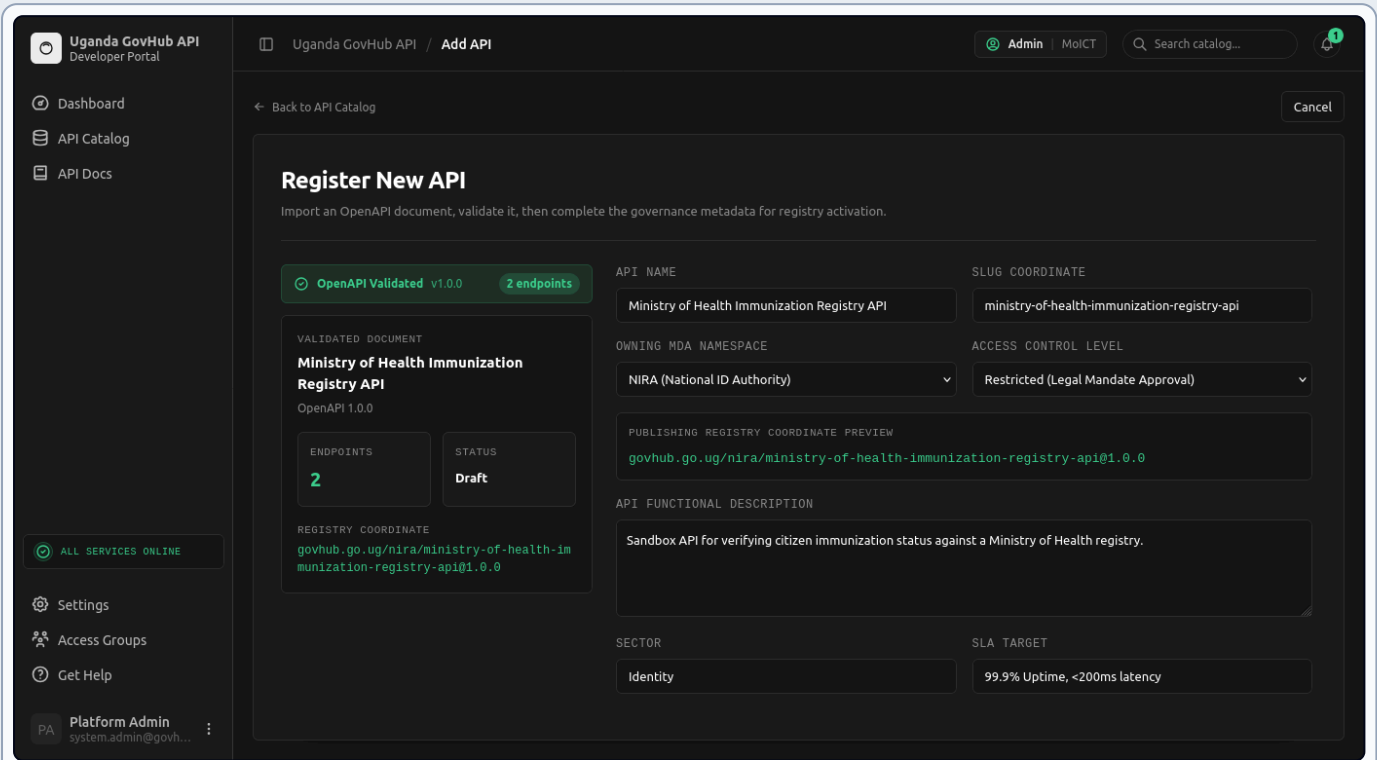
Machine-readable contracts reduce manual documentation drift.

Screenshot evidence: API import

Page 12 of 18

Governance metadata capture

After validation, the registry flow captures owner namespace, access level, functional description, sector, SLA target, and registry coordinate preview.



METADATA

The validation result extracts title, version, endpoint count, namespace, access level, description, sector, SLA, and registry coordinate preview.

Ownership

Owning MDA namespace and access control level are required registry attributes.

Service quality

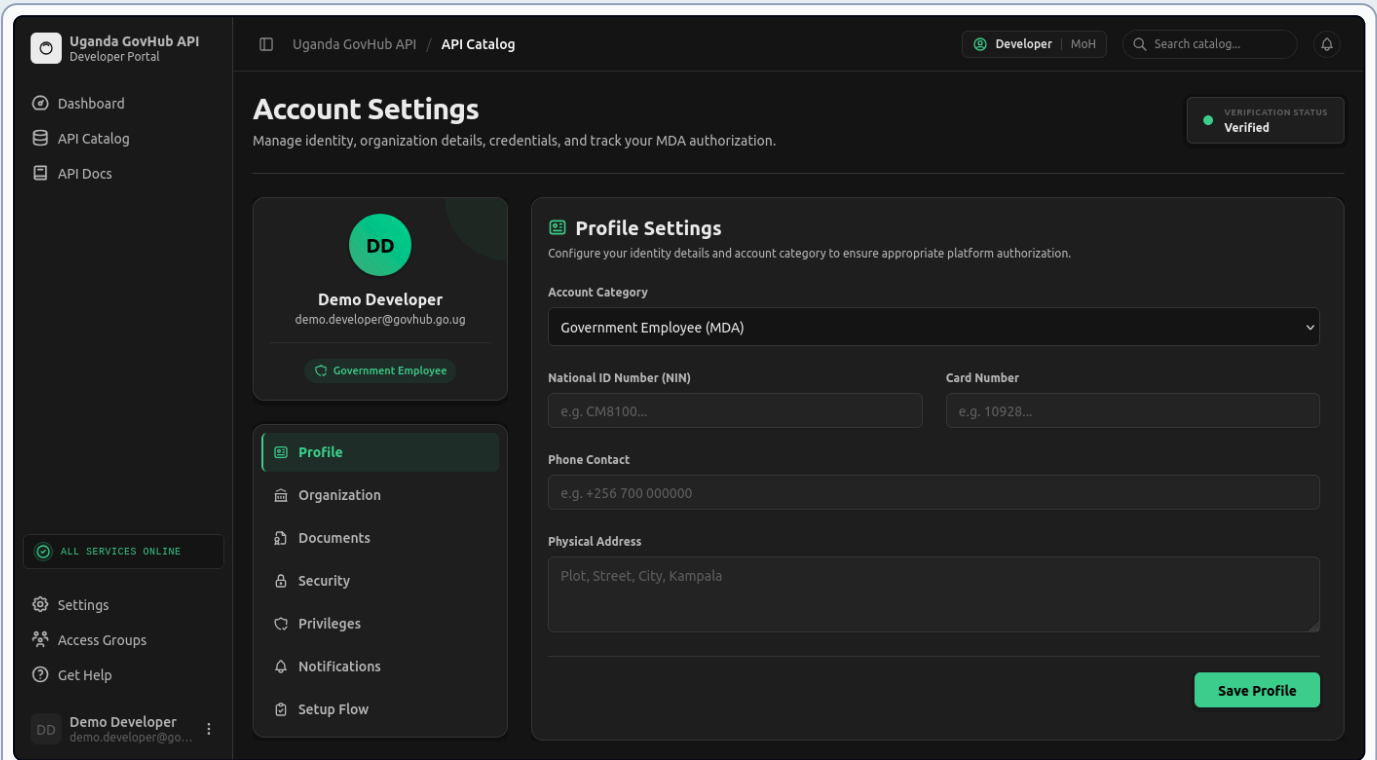
SLA target and lifecycle status support operational review.

Registry coordinate

The API receives a predictable coordinate for catalog and documentation linkage.

Account settings and verification state

The account settings surface shows how user identity and MDA authorization are managed before operational API access is granted.



ACCOUNT SETTINGS The verified developer account demonstrates account category, national ID/card placeholders, profile controls, verification status, and access-group readiness for platform authorization.

Identity

Profile and account category fields support reviewable user identity.

Verification

Verified status is visible in the account surface and top-level shell.

Privileges

Organization, documents, security, privileges, and setup flow are separated.

How data moves through the prototype

The core flow starts with API discovery and ends with an audited sandbox transaction. Access is explicit, credentials are scoped, and every allowed or denied sandbox call can be traced by correlation ID.

1

DISCOVER

2

REQUEST

3

APPROVE

4

AUDIT

Stored data categories

- Catalog and governance metadata for each MDA API product.
- OpenAPI contracts and version records.
- User, session, account verification, role, and MDA assignment records.
- Access requests, lawful basis, requested fields, expiry, revocation state, and API key hashes.
- Audit events with MDA/API context, actor, event type, correlation ID, timestamp, and redacted metadata.

Implemented security controls

- Transport security hooks for TLS/HSTS and CORS allow-listing.
- Password hashing, MFA support, role-based access, and verified account states.
- AES-256-GCM sensitive-field encryption support and hash-only API key storage.
- Scoped sandbox keys, endpoint authorization, expiry, revocation, and rate limits.
- Server-generated correlation IDs, audit logs, and sensitive-value redaction.

No live citizen, tax, business, or permit records are exchanged. Sandbox handlers return deterministic mock responses that demonstrate interoperability behavior without exposing production government data.

APIs represented in the prototype

The seeded catalog demonstrates how several MDAs could publish governed, documented services through one discovery and sandbox experience. The implementation uses mock handlers only.

OWNER	API PRODUCT	PRIMARY INTEROPERABILITY VALUE	CLASSIFICATION
NIRA	Identity Verification	Verify identity match and return the minimum fields needed for a lawful service workflow.	Restricted
URA	Tax Compliance Status	Check tax-clearance style status for eligibility, procurement, and compliance workflows.	Official
URSB	Business Registration Lookup	Validate business registration facts, current status, and registry details.	Public
MoWT	Driving Permit Verification	Confirm permit validity, class authorization, expiry, and suspension state.	Official
MoICT	Service Uganda Composite Eligibility	Show multi-agency orchestration using approved sandbox scopes and correlation IDs.	Restricted

Current prototype

Requests terminate inside the Express backend. Middleware validates key status and scope, then route handlers return deterministic mock responses and structured errors.

Production pathway

A production deployment would connect through approved national integration infrastructure or MDA gateways, with contract testing, adapter services, message signing, monitoring, and data-sharing agreements.

Owner accountability

Each API keeps an owning MDA, technical owner, approval authority, statutory basis, purpose limitation, retention class, and audit requirement.

External-system statement: NIRA, URA, URSB, MoWT, NITA-U, and other production government systems are not integrated in this prototype. They are represented through realistic mock contracts and sandbox handlers.

Integration control model

GovHub is positioned as a governed API portal and sandbox layer around MDA-owned services. The production version would preserve agency ownership while standardizing documentation, approval, credential control, monitoring, and audit visibility.

01

MDA publishes API contract and governance metadata.

02

Consumer discovers the API and submits lawful access purpose.

03

Owner or reviewer approves scoped access and expiry.

04

Sandbox call executes with key scope and rate-limit checks.

05

Audit trail records actor, MDA, API, event, and correlation ID.

Path from showcase prototype to government deployment

The local prototype is intentionally small, but the architecture identifies where production hardening would be applied: runtime isolation, database hosting, secret management, monitoring, backup, and approved integration channels.

Current hosting model

Frontend runs as a Vite React application. Backend runs as an Express service, defaulting to localhost. PostgreSQL and OpenAPI files support the repeatable local demonstration.

Production hardening path

Serve static assets behind approved hosting, run backend services in containers or managed application services, use managed PostgreSQL, centralized logs, backups, KMS-backed secrets, and approved integration gateways.

Scalability

The REST API is stateless apart from database and file-storage dependencies. Multiple backend instances can serve traffic behind a load balancer when shared storage is centralized.

Resilience

Production should use managed relational storage, backups, disaster recovery, health checks, deployment rollback, and separate secret management.

Security operations

Move sandbox access toward key rotation, replay protection, stronger signing, centralized logs, monitoring alerts, and tamper-evident audit anchoring where required.

Runtime

Containerized backend services behind approved ingress, health checks, and rollout controls.

Data

Managed PostgreSQL, backups, retention policy, encryption keys, and recovery procedures.

Secrets

KMS-backed environment secrets, key rotation, least-privilege service credentials.

Observability

Central logs, metrics, request traces, audit export, and incident alerts.

Production readiness sequence

- Replace local development servers with hosted frontend assets and load-balanced backend runtime.
- Move OpenAPI storage and PostgreSQL persistence to managed, backed-up infrastructure.
- Integrate approved national or MDA gateway adapters behind explicit data-sharing agreements.
- Run contract tests, access-control tests, audit-log checks, vulnerability scanning, and operational rehearsal before pilot use.

Coverage of the Ministry architecture guidance

The document explicitly maps system components, user access, storage, security, data flow, APIs, integrations, hosting, and future readiness to the requested architecture submission elements.

GUIDANCE ITEM	WHERE IT IS ADDRESSED IN THIS SUBMISSION
Main components	Architecture diagram, component inventory, and platform screenshots show the frontend, backend, PostgreSQL database, OpenAPI storage, sandbox services, and production boundary.
Communication and protocols	Browser access, REST/JSON API calls, scoped sandbox key checks, HTTPS/TLS posture, and correlation IDs are described across the architecture and security sections.
Data storage and security	PostgreSQL categories, OpenAPI files, encrypted fields, key hashing, rate limits, redaction, and audit logging are covered in the component and data-flow sections.
User access	Public visitors, verified developers, API owners, reviewers, administrators, account settings, and account review states are shown in text and screenshots.
External integrations	The integration section identifies NIRA-style, URA-style, URSB-style, MoWT-style, and composite sandbox APIs, and clearly states production systems are not called.
Hosting and infrastructure	The current local environment is distinguished from the production hardening path: load-balanced services, managed Postgres, secret storage, backups, logs, and monitoring.

This PDF is intended as an architecture submission for the Government Systems Prototype Showcase. It communicates system substance: components, data movement, operational controls, security posture, integration boundaries, and visible platform workflows.

Components

Frontend, backend, database, OpenAPI storage, sandbox services, and future integration boundary.

Security

Authentication, account verification, RBAC, key hashing, encryption support, rate limits, redaction.

Workflows

Discovery, docs, access request, approval, sandbox execution, matrix review, API onboarding.

Readiness

Production hosting path, managed database, secrets, monitoring, backups, and gateway integration.

Evaluator takeaway

The prototype is not a live national integration platform and does not exchange production citizen data. It is a working architecture model for how MDA APIs can be made discoverable, governed, documented, requestable, testable, and auditable through a common developer portal and sandbox.

The architecture is intentionally transferable: React and TypeScript on the frontend, Node.js and Express on the backend, OpenAPI as the contract format, PostgreSQL for persistence, and standard HTTP security controls for deployment hardening.